

Assignment-1 — MAXSAT / SAT

Rodrigo Guerreiro nº 75739 · Dinis Custódio nº 79895

Os pontos 1 a 4 do enunciado correspondem a tarefas cujos resultados são entregues nos ficheiros do ZIP (`hoos.cnf` e `sat_solver.py`) e no output do programa. A listagem pedida no ponto 4 é 11000 ($x_1 = x_2 = \text{True}$, $x_3 = x_4 = x_5 = \text{False}$). As questões com resposta escrita são a 5 e a 6, apresentadas de seguida.

1. Questão 5

Run your algorithm on the `uf20-01.cnf`, `uf20-02.cnf`, and `uf20-03.cnf` instances. How many solutions satisfy each problem instance?

Usámos as três primeiras instâncias: `uf20-01`, `uf20-02` e `uf20-03`. O número de soluções de cada instância é:

Instância	Variáveis	Cláusulas	Nº de soluções	Tempo de busca (s)
<code>uf20-01.cnf</code>	20	91	8	3,14
<code>uf20-02.cnf</code>	20	91	29	3,02
<code>uf20-03.cnf</code>	20	91	1	3,02

Tabela 1: Número de soluções e tempo de busca medido pelo programa.

Cada execução visita $2^{20} = 1\,048\,576$ atribuições e avalia 91 cláusulas por atribuição com curto-circuito, isto é, a avaliação de uma cláusula para no primeiro literal verdadeiro e uma atribuição é descartada na primeira cláusula falsa. O tempo de busca é medido pelo próprio programa (ver Questão 6). Por exemplo, a instância `uf20-03` tem uma única solução:

```
11110111111010011101
```

2. Questão 6

It is impractical to do the same for a larger instance, say one contained in `uf100-430`. How could you estimate the time needed by your computer program to complete the task on such an instance? Justify your answer.

O programa visita exaustivamente todas as 2^n atribuições, pelo que o tempo cresce exponencialmente com o número de variáveis n . O tempo da fase de busca é medido com `time.process_time()` e reportado na linha de timing do formato DIMACS (`satformat.pdf`, secção 2.4):

```
t cnf <SOLUTION> <VARIABLES> <CLAUSES> <CPUSECS> <MEASURE1>
```

em que `MEASURE1` = 2^n é o número de atribuições visitadas — uma medida algorítmica e reproduzível, independente da máquina. Exemplo real (`uf20-01`):

```
t cnf 1 20 91 3.141 1048576
```

A estimativa para $n = 100$ faz-se extrapolando esse valor medido:

- `uf100-430` tem $n = 100$ variáveis, logo $2^{100} \approx 1,27 \times 10^{30}$ atribuições.
- Fator de crescimento face a $n = 20$: $2^{100}/2^{20} = 2^{80} \approx 1,21 \times 10^{24}$.
- Tempo medido para $n = 20$: $T(20) \approx 3,1$ s (mínimo de 3 execuções; os tempos variam com a carga da máquina).

- $T(100) \approx 3,1 \times 2^{80} \text{ s} \approx 3,7 \times 10^{24} \text{ s} \approx 1,2 \times 10^{17} \text{ anos}$.

Justificação:

- A complexidade é $\Theta(2^n \cdot m)$. O corte ao primeiro literal/cláusula falsa (curto-circuito) e o aumento do número de cláusulas ($91 \rightarrow 430$) são apenas fatores constantes e não alteram o crescimento exponencial.
- Mesmo com um computador 10^9 vezes mais rápido, o tempo continuaria a ser cerca de $1,2 \times 10^8$ anos, pelo que a força bruta é inviável para $n = 100$.
- Para instâncias maiores seria necessário substituir a enumeração exaustiva por métodos que explorem a estrutura do problema, como retrocesso com propagação (DPLL), *solvers* CDCL ou heurísticas de procura local no contexto MAXSAT.